

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Цветкова Татьяна

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург 2026 г.

Задача

Тема: Домашнее задание №4. Технический дизайн микросервисной архитектуры.

Вариант: «Платформа для фитнес-тренировок и здоровья».

Цель работы: разделить монолитное бэкенд-приложение (LP1) на микросервисы по принципу database-per-service, спроектировать способы их взаимодействия, разделение БД и межсервисные эндпоинты в формате OpenAPI; описать конкретные требования и шаги миграции из монолита.

Задание:

1. Разделить монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service.
2. Спроектировать архитектуру: отразить взаимосвязь между сервисами и описать способы взаимодействия.
3. Спроектировать разделение БД на обособленные БД.
4. Описать межсервисные эндпоинты в формате OpenAPI с телами запросов, ответов и возможными ошибками.
5. Сформировать документ с конкретными требованиями и шагами для разделения монолита на микросервисы.

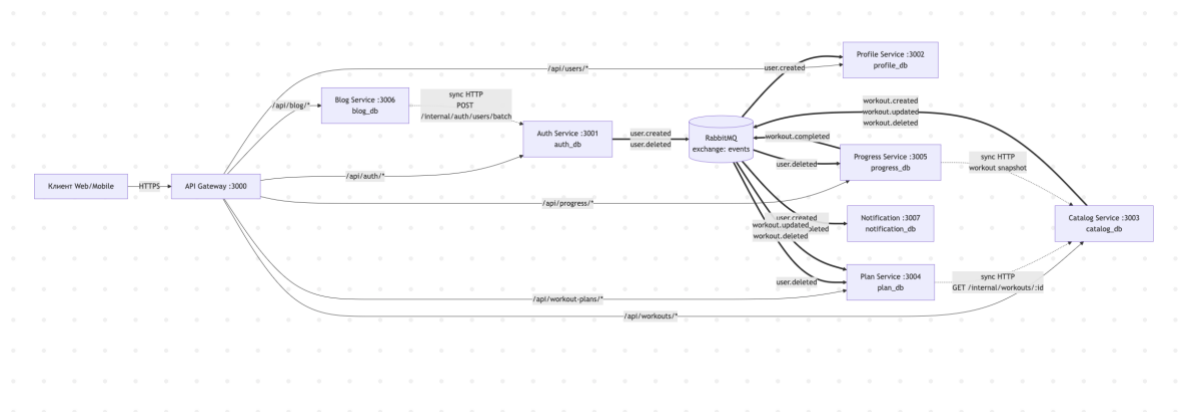
Ход работы

Исходный монолит реализован на Express + TypeORM + TypeScript (LP1) и содержит девять сущностей: User, WorkoutCategory, Workout, WorkoutPlan, PlanItem, ProgressEntry, BlogCategory, BlogPost, BlogComment. По границам бизнес-доменов выделены семь предметных микросервисов плюс API Gateway и message broker.

1. Карта сервисов

- api-gateway (порт 3000) — маршрутизация, валидация JWT, rate limit, без БД;
- auth-service (3001) — регистрация, вход, выдача access и refresh JWT, ролевая модель; БД auth_db;

- profile-service (3002) — антропометрия, уровень подготовки, аватар; БД profile_db;
- catalog-service (3003) — тренировки и категории, поиск и фильтрация; БД catalog_db;
- plan-service (3004) — личные планы тренировок и их элементы; БД plan_db;
- progress-service (3005) — трекинг выполненных тренировок и сводная статистика; БД progress_db;
- blog-service (3006) — статьи о здоровье и питании, категории, комментарии; БД blog_db;
- notification-service (3007) — email/push уведомления (опциональный);
- rabbitmq (5672) — message broker для асинхронного взаимодействия.



2. Разделение БД (database-per-service)

Каждый сервис владеет собственной PostgreSQL-базой. Сущность User монолита распадается на две: auth_db.users (id, email, username, password_hash, role) и profile_db.user_profiles (user_id, first_name, last_name, birth_date, weight_kg, height_cm, fitness_level, avatar_url). Каталог тренировок и категории переносятся целиком в catalog_db.workouts и catalog_db.workout_categories. Личные планы — в plan_db.workout_plans и plan_db.plan_items, причём элементы плана хранят денормализованные snapshot-поля (workout_title, workout_duration_min, workout_type) — это позволяет показывать план, даже если catalog-service временно недоступен или тренировка удалена. Записи прогресса — progress_db.progress_entries

— также хранят `workout_title_snapshot`. Блог — `blog_db.blog_posts`, `blog_db.blog_categories`, `blog_db.blog_comments` — хранит `author_display_name` денормализованно. Прямого доступа к чужим БД ни один сервис не имеет; обмен данными — только через HTTP API и события.

3. Способы взаимодействия

Синхронные межсервисные вызовы реализованы по REST/HTTP в служебной сети (mTLS + общий служебный токен X-Service-Token). Применяются там, где ответ нужен немедленно: API Gateway вызывает `auth-service` для валидации JWT (либо валидирует stateless по публичному ключу); `plan-service` при добавлении тренировки в план дергает `GET /internal/workouts/{id}` у `catalog-service`, чтобы проверить существование и получить `snapshot`; `progress-service` при создании записи прогресса аналогично запрашивает данные тренировки; `blog-service` делает batch-запрос `POST /internal/auth/users/batch` для обогащения авторов постов и комментариев. На все внешние HTTP-вызовы накручен `circuit breaker`, `timeout 2 c` и экспоненциальный `retry` для идемпотентных операций.

Асинхронные взаимодействия идут через RabbitMQ, `exchange events`. Описаны события: `user.created` (`auth` → `profile`, `notification`), `user.deleted` (`auth` → `profile`, `plan`, `progress`, `blog`), `workout.created/updated/deleted` (`catalog` → `plan`, `progress`), `workout.completed` (`progress` → `notification`). Гарантии доставки — `at-least-once` с дедупликацией на стороне consumer-а по `eventId`; на producer-стороне применяется `Outbox-pattern`: событие пишется в одной транзакции с бизнес-данными в локальную таблицу `outbox`, отдельный `worker` публикует его в брокер. Сообщения, обработка которых упала несколько раз, попадают в `dead-letter queue` для ручного разбора.

4. Межсервисные эндпоинты (OpenAPI)

Полная спецификация internal-эндпоинтов и схем событий — в файле `microservices.openapi.yaml`, прилагаемом к работе. Ключевые эндпоинты:

- `POST /internal/auth/validate` — принимает `{token}`, возвращает `{valid, userId, email, role, expiresAt}` или 401 при невалидном токене;
- `GET /internal/auth/users/{userId}` и `POST /internal/auth/users/batch` — базовая информация о пользователе(ях) для блога и аналитики;
- `GET /internal/profiles/{userId}` и `POST /internal/profiles/batch` — публичный профиль для отображения в выдаче и шапке;

- GET /internal/workouts/{id} и POST /internal/workouts/batch — для plan-service и progress-service при denormalization;
- GET /internal/progress/by-user/{userId}/stats — сводная статистика прогресса для отображения в профиле.

Возможные коды ответов: 200 OK, 400 Bad Request (невалидный payload), 401 Unauthorized (отсутствует или невалиден X-Service-Token), 404 Not Found, 409 Conflict (например, дубль user.created), 422 Validation Error, 500 Internal Server Error, 503 Service Unavailable (при перегрузке). Все ошибки возвращаются в едином формате {error: string, details?: any}.

5. Аутентификация в распределённой системе

Принят stateless-подход: auth-service подписывает JWT приватным ключом по алгоритму RS256 и публикует public key через эндпоинт /jwks.json. API Gateway и downstream-сервисы валидируют токен локально по публичному ключу без сетевых вызовов. Эндпоинт /internal/auth/validate сохранён как fallback на случай немедленного отзыва токена. Gateway прокидывает заголовки X-User-Id и X-User-Role в downstream, чтобы те не валидировали токен повторно.

6. Шаги миграции из монолита (Strangler Fig)

Шаг 1. Поднять API Gateway (nginx или Express-прокси) перед монолитом, переключить клиентов на него — без изменения логики, только маршрутизация.

Шаг 2. Выделить auth-service. Завести auth_db, перелить таблицу users. Gateway маршрутизирует /api/auth/* в новый сервис, остальное — в монолит. Переключить выдачу JWT на RS256 с публичным ключом.

Шаг 3. Поднять RabbitMQ. Включить публикацию user.created из auth-service через Outbox-pattern.

Шаг 4. Выделить profile-service. Создать profile_db, перенести антропометрические поля из user монолита, подписаться на user.created/user.deleted.

Шаг 5. Выделить catalog-service. Read-heavy и слабо связан — простой кандидат. Поднять catalog_db, перенести workouts и

database-per-service; данные, ранее лежавшие в одной таблице User, разделены между auth_db и profile_db.

Описаны два типа взаимодействия: синхронные REST/HTTP-вызовы между сервисами (с mTLS, circuit breaker, retry и таймаутами) — для случаев, когда ответ нужен немедленно; и асинхронные события через RabbitMQ — для слабосвязанной интеграции. Применены паттерны Outbox для надёжной публикации событий, denormalization snapshot-полей для отказоустойчивости и Strangler Fig для постепенной миграции.

Все межсервисные эндпоинты задокументированы в формате OpenAPI 3.0 в прилагаемом файле microservices.openapi.yaml: тела запросов и ответов, security schemes, схемы событий брокера и возможные ошибки (400, 401, 404, 409, 422, 500, 503). Сформирован пошаговый план миграции из монолита в 10 этапов с указанием порядка выделения сервисов и стратегии отката. Цель работы достигнута: получен документ, в котором описаны конкретные требования и шаги для разделения монолитного бэкенда на отдельные микросервисы.